CASE STUDY REPORT

Campus Lost-and-Found Management System

Using C++ and Object-Oriented Programming

Real-world Problem Analysis | Proposed Solution | C++ OOP

LOST & FOUND RECORDS

ITEM	LOCATION	STATUS
Wallet	Library	LOST
Identity card	Cafeteria	FOUND
Water bottle	Laboratory	RETURNED
Keys	Hostel	LOST
Textbook	Classroom	FOUND

SUBMITTED BY

Name Kavya

UID 25BCD10086

Course BCA (Data Science)

Subject Object-Oriented Programming in C++

Case study type: real-world problem analysis and proposed software solution

01 Case Study Overview

CASE STUDY TITLE	SUBJECT	CASE STUDY TYPE	LANGUAGE
Campus Lost-and-Found Management System	**Object-Oriented Programming in C++**	**Problem analysis and proposed solution**	**C++**

Contents

01	Case Study Overview	2	07	Software and Hardware Requirements	5
02	Abstract	2	08	OOP Concepts Used	5
03	Introduction	3	09	System Design and Workflow	7
04	Problem Statement	3	10	Benefits, Challenges and Future Improvements	8
05	Objectives	4	11	Conclusion	8
06	Scope of the Case Study	4			

02 Abstract

A **Campus Lost-and-Found Management System** is a proposed software application designed to help students, faculty members, and other campus visitors report lost belongings and register items they have found. In educational institutions, students frequently lose items such as identity cards, wallets, books, keys, water bottles, and electronic accessories. Traditional methods of reporting lost items, such as notice boards, classroom announcements, or contacting security staff, may be inefficient and difficult to maintain.

This case study examines how a computer-based lost-and-found system can improve the reporting, searching, and recovery of lost belongings. The proposed system allows users to register lost and found items, enter descriptions and locations, search for matching records, and update the status of an item after it has been returned.

The solution is designed around C++ and **object-oriented programming** principles. Classes and objects represent items and manage records, encapsulation protects the internal data structure, and file handling can provide persistent storage. The case study also discusses the benefits, limitations, implementation challenges, and future improvements of the proposed system.

The main purpose is to demonstrate how OOP concepts can be applied to a practical campus problem while improving organization, accessibility, and efficiency.

03 Introduction

A college campus contains classrooms, laboratories, libraries, cafeterias, playgrounds, hostels, and other shared spaces. Because students and staff move between these locations throughout the day, personal belongings can sometimes be misplaced or accidentally left behind.

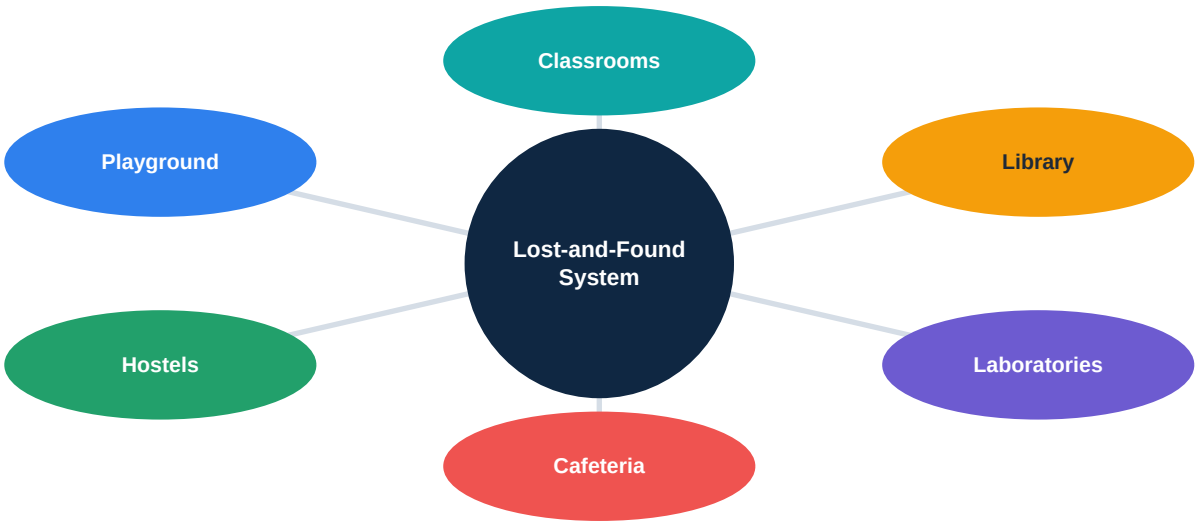
When someone loses an item, they may ask classmates, contact teachers, visit the security office, or search notice boards. Similarly, a person who finds an item may not know how to identify its owner or report it to the appropriate authority. These disconnected methods can cause delays, duplicate reports, and difficulty tracking recovered belongings.

A Campus Lost-and-Found Management System provides a centralized way to manage these situations. Users can record details about lost or found items, search existing reports, and check whether a potential match exists. Authorized campus staff could also help verify ownership before returning valuable items.

Object-oriented programming is suitable for this system because it allows related data and operations to be organized into classes. For example, an **Item** class can store the name, category, description, location, and status of an item. A management class can provide operations for registering, searching, displaying, and updating records.

This case study explains the problem, proposes a structured software solution, and demonstrates how C++ OOP concepts can support its implementation.

Where items go missing on campus



04 Problem Statement

The existing process of reporting lost and found belongings on a college campus is often informal and decentralized. Information may be shared through verbal communication, messaging groups, classroom announcements, or physical notice boards. As a result, important details may be missed, records may become outdated, and students may find it difficult to determine whether a missing item has been recovered.

Problem: the absence of a centralized, organized method for recording and searching lost and found items within a campus.
Proposed solution: a Campus Lost-and-Found Management System that maintains item records in a structured format. Users can report lost belongings, register found objects, search by keyword or category, and update the recovery status when an item is returned.

Today: informal and scattered

- Verbal messages and chat groups
- Notice boards that go out of date
- Duplicate or missed reports
- Unclear if an item was recovered

improve

Proposed: centralized system

- One structured record per item
- Search by keyword or category
- Status: lost, found or returned
- Possible matches shown quickly

05 Objectives

1. Study the challenges of managing lost belongings on a college campus
2. Propose a centralized system for lost and found records
3. Allow users to register lost and found items with relevant details
4. Provide searching and matching functionality
5. Track whether an item is lost, found, or returned
6. Apply classes, objects, encapsulation and other OOP concepts
7. Explore file handling for persistent storage
8. Reduce the time and effort needed to recover misplaced belongings
9. Identify limitations and possible future improvements

06 Scope of the Case Study

The proposed system is intended for colleges, universities, hostels, libraries, and other educational institutions.

Included in scope

- Registering item descriptions
- Recording dates and locations
- Searching item records
- Identifying possible matches
- Displaying results
- Updating item status

Possible extensions

- Web interface
- User authentication
- Notifications
- Administrator dashboard

Limitations

- Ownership is not verified automatically
- A matching description does not prove ownership
- Physical verification and checks by authorized campus staff are still necessary

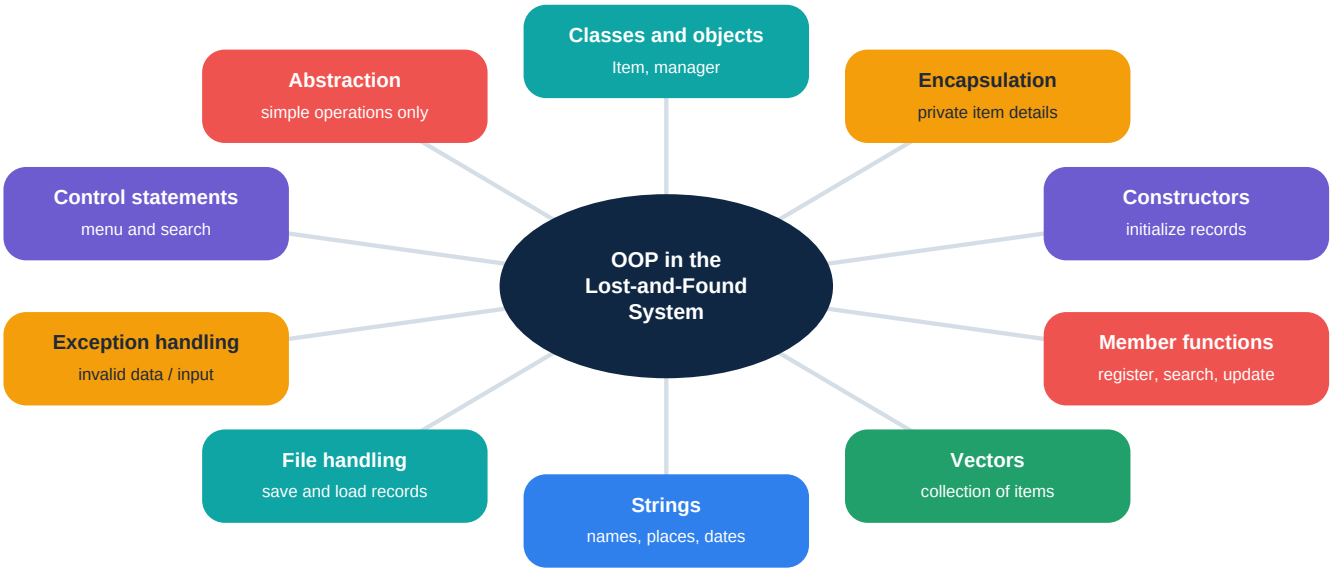
07 Software and Hardware Requirements

Component	Requirement
Programming language	C++
Compiler	GCC or Clang supporting C++17
Code editor	Visual Studio Code
Operating system	macOS, Windows, or Linux
Storage	Local text file
Libraries	Standard C++ library
Hardware	Basic computer capable of running a C++ compiler

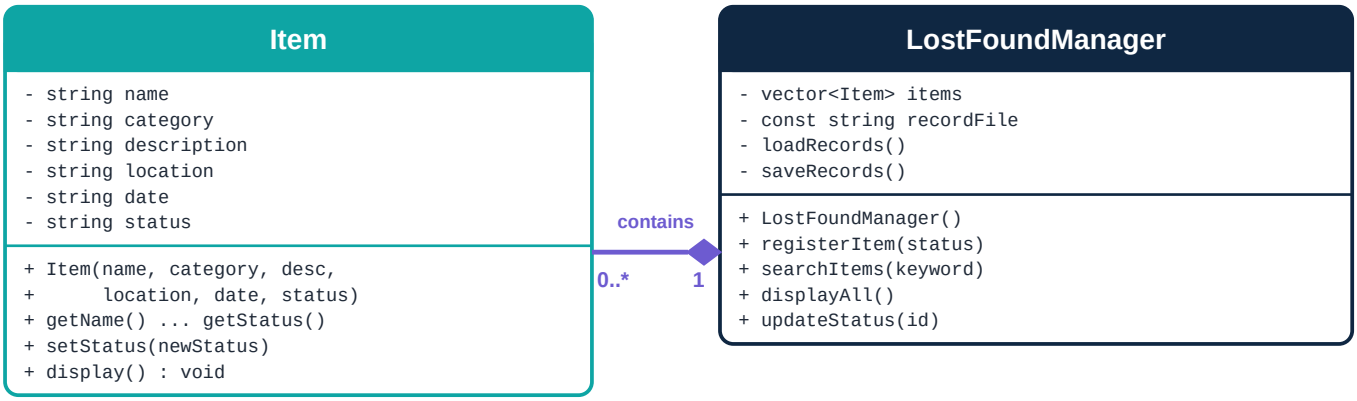
08 OOP Concepts Used

Concept	Application
Classes and objects	Represent campus items and the management system.
Encapsulation	Keeps item details private and exposes controlled operations.
Constructors	Initialize item records with their details.
Member functions	Register, display, search, and update item records.
Vectors	Maintain collections of item records during execution.
Strings	Store names, descriptions, locations, categories, and dates.
File handling	Save and retrieve item records.
Exception handling	Handle invalid stored data or input conversion errors.
Control statements	Manage the menu and search operations.
Abstraction	Provide simple operations without exposing internal storage details.

OOP concept map



Proposed class design (UML)

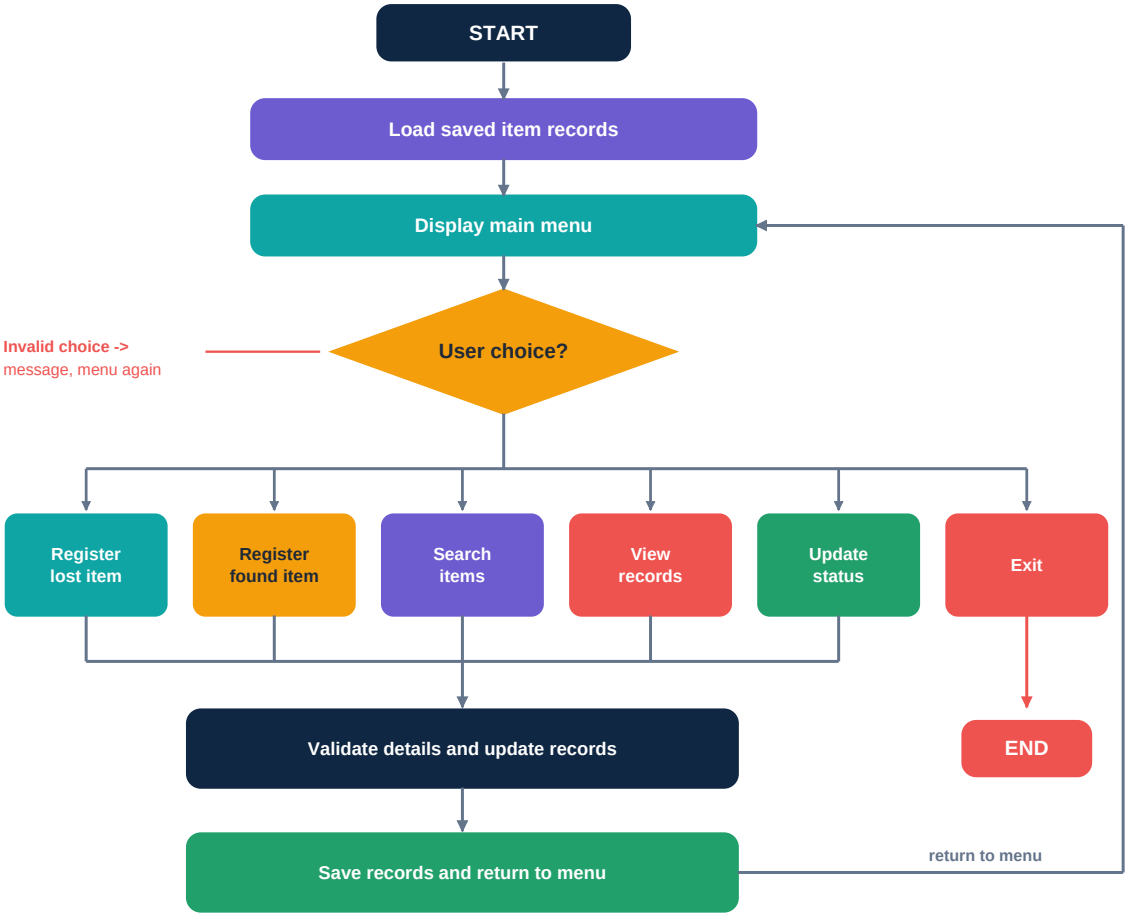


Proposed design: an Item stores one record; the manager keeps a vector of Items and offers register, search, display and update.

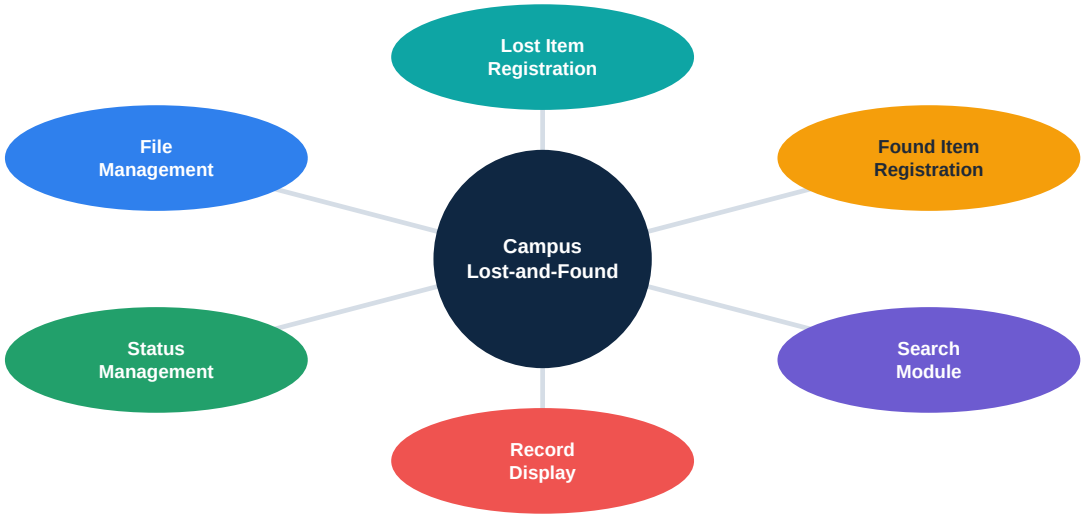
- private member (encapsulated) + public member (interface) <>-- composition (manager owns many Items)

09 System Design and Workflow

The application starts by loading saved item records, displays a main menu, performs the chosen operation, validates the details, updates the records, saves them, and returns to the menu.

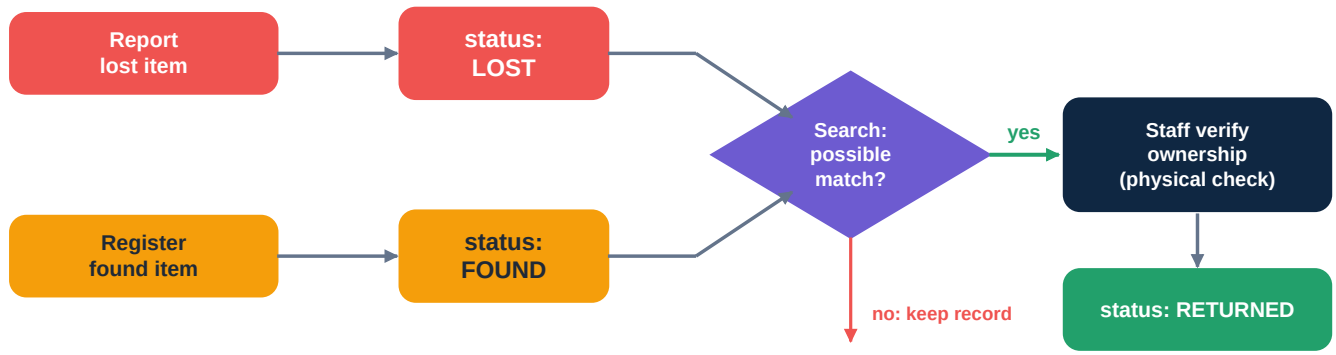


Main modules



Module	Responsibility
Lost Item Registration	Records information about belongings reported missing.
Found Item Registration	Stores information about items discovered on campus.
Search Module	Searches records using item names or categories.
Record Display	Shows all registered items and their current status.
Status Management	Updates a record when an item is recovered or returned.
File Management	Maintains records between program sessions.

Life cycle of an item record



10 Benefits, Challenges and Future Improvements

Benefits

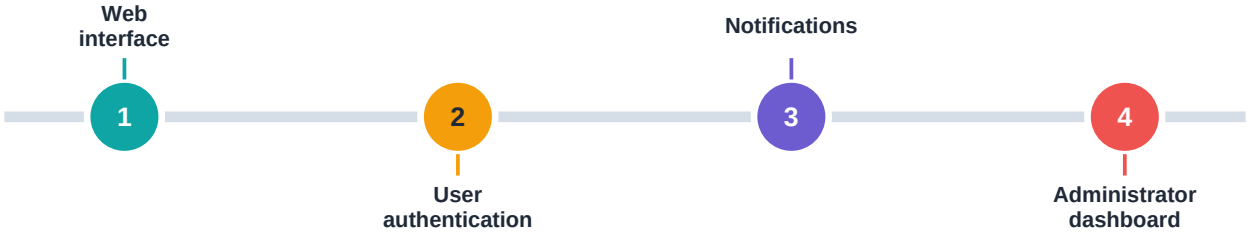
- One central, organized record of items
- Faster searching by keyword or category
- Clear status: lost, found or returned
- Less duplication and fewer missed reports

Challenges

- Ownership cannot be verified automatically
- Staff must verify items physically
- Records must be kept up to date
- Users must enter clear descriptions

Future improvements

- Web interface
- User authentication
- Notifications
- Administrator dashboard



11 Conclusion

Lost belongings are a common problem on a busy campus, and informal reporting methods make recovery slow and uncertain. A Campus Lost-and-Found Management System offers a centralized, searchable and organized alternative. By modelling items and the management of records with classes, objects, encapsulation, vectors, strings and file handling, the proposed C++ solution shows how OOP concepts apply to a practical campus problem. Although matching descriptions cannot replace physical verification by authorized staff, the system can greatly improve organization, accessibility and efficiency, and it can grow into a web-based service with authentication, notifications and an administrator dashboard.